

Milestone 1

Proposal submission

Due April 10

- Finalize project proposal document

Polish the proposal generated earlier — add your name, course code, and submit as PDF to professor.

- Draw and include architecture diagram

Embed the SVG architecture diagram in the proposal. Professors love seeing this at proposal stage.

- Create GitHub repository

Set up repo with README, folder structure (src/, docs/, infra/, dashboard/), and .gitignore. Link in proposal.

Milestone 2

Literature map + baseline code

Due April 24

- Literature review — IoT healthcare monitoring

Find 8-10 papers on assistive IoT monitoring systems. Google Scholar: "IoT elderly monitoring", "cloud assistive healthcare".

- Literature review — IoT security

Find 5-8 papers on MQTT security, TLS in IoT, device authentication. Search "MQTT TLS IoT security", "zero trust IoT".

- Write bibliography report (15-20 pages)

Summarize each paper, map to your system layers, identify research gaps. Include your architecture as context.

- Set up Raspberry Pi + Azure IoT Hub

Install Pi OS, register device in Azure IoT Hub (F1 free tier), install azure-iot-device SDK, send first test message.

- Write baseline sensor simulation script

If hardware not ready: simulate PIR + button + DHT22 values in Python and send to IoT Hub. Counts as baseline code.

Milestone 3

Methodology

Due May 15

- Connect physical sensors to Raspberry Pi

Wire PIR sensor, push button, DHT22 to GPIO. Test each with individual Python scripts before integrating.

- Implement sensor_reader.py

GPIO polling loop running as background thread. Outputs events to an internal queue consumed by alert engine.

- Implement alert_engine.py

Inactivity timer logic, temperature range checks, emergency button handler. Generates severity-tagged alert events.

- Implement azure_client.py with TLS

Wrap Azure IoT SDK. Send telemetry over MQTT/TLS. Handle reconnection. Load connection string from config, never hardcoded.

- Set up Azure Table Storage

Create storage account. Write storage_writer.py to receive IoT Hub messages and persist rows (partitionKey=deviceId, rowKey=timestamp).

- Write methodology section of report

Explain hardware choices, software architecture, cloud pipeline, and security design decisions. Reference your architecture diagram.

Milestone 4**Demo — experiments and analysis**

Due June 12

- **Build Streamlit dashboard** Progress
Live sensor status, activity timeline chart, alert log table. Pulls from Azure Table Storage every 10s via `st.rerun()`.
- **Implement alert notifications** Progress
Email or Teams webhook triggered on critical events (inactivity, emergency button, temperature out of range).
- **Implement threat_sim.py** Progress
Script that attempts connection with invalid SAS key. Logs the 401 rejection. Visualize failed attempts on dashboard.
- **Add dashboard access control** Progress
Minimum: password gate via `st.secrets`. Discuss RBAC conceptually in report (caregiver / admin / read-only roles).
- **Run and document experiments** Progress
Test inactivity detection, emergency trigger, temp alert, unauthorized device rejection. Capture results as tables or charts.
- **Record demo video** Progress
Screen + hardware capture showing: sensor → cloud → dashboard → alert → threat rejection. 3–5 minutes max.

Milestone 5**Final report + presentation**

Due July 3

- **Write full project report (12–20 pages)** Progress
Sections: introduction, literature review, system design, methodology, implementation, security analysis, results, conclusion.
- **Write cybersecurity analysis section** Progress Progress
Cover TLS, device auth, RBAC, threat simulation results. Frame around your research question on securing patient data.
- **Prepare presentation slides (10–12 min)** Progress
Architecture diagram, demo screenshots, key results, security findings, lessons learned. Keep slides sparse — let the demo talk.
- **Prepare 1-page handout** Progress
Required by course. System overview, key results, GitHub link, tech stack. One side of A4.
- **Finalize GitHub repo for submission** Progress
Clean README with setup instructions, architecture diagram, module descriptions, how to run demo. Reproducibility is graded.

Ongoing**Throughout the project**

- **Commit code to GitHub regularly** Progress
Commit after every working feature. Shows progress history and satisfies the reproducibility requirement.
- **Keep a process log** Progress
Brief notes on what you built, what broke, and how you fixed it. This feeds directly into the Protokoll (process documentation) component worth 30%.